

1. Introduction.

The CFS project was developed for solution of partial differential equations by means of finite element as well as boundary element method. It was projected, keeping in mind to use it for the implementation of adaptivity for FE/BE methods. Another important criteria for design of this code is ability to solve coupled field problems. Code of CFS was implemented on C++ programming languages due to object-oriented concept of it. Hierarchy of classes, virtual classes, mechanisms of templates play an important role throughout implementation. For the code development we used GNU C++ compiler(version 2.95) on LINUX and CC on SGI. At present scalar problem such as acoustic equation can be treated, It is solved by using the Newmark method.

We give a sketch of design of program and some issues of implementation.

2. Design criteria .

- *separation of topological and algebraic entities*

It is an important criteria for design of code for implementation of adaptivity due to complexity of work with grid. By using global node-numbering strategy we are able to transfer structure of grid into algebraic structure.

- *clear structure of code*

It should be easy to catch the structure of the program for users to be able to implement its own practical problems. So it should be designed in the way to hide and shield some structures and data from users.

- *efficiency*

At the same time it should be efficient, because computations are often time-consuming, especially in 3D. Of course, this and former criteria are contradictory and each time we have to trade against each other.

- *features of coupled field problems*

One of the main goal is to provide a program for the general handling of coupled field problems. Therewith, matrix as well as right hand side coupling will be supported.

3. Structure of code.

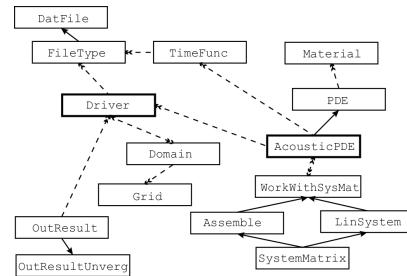


Fig. 1. Structure of CFS

The main classes of code are shown on the figure. We give a short description of these classes.

4. Description of classes.

4.1. Classes for input and output data

- Class **DefineInOutFiles**
In this class we define auxiliary files, such as *trace* -file (file, where we list all methods and classes, which were used during running of program; this file should help developer to trace a mistake), *debug* -file (file, where intermediate results are stored), *info* -file (in this file we print specific information about methods and types of data, which were used in code)
- Class **FileType**
Base class for reading input data. Its function is virtual due to handle the different types of input files and to hide their features from developer of code.
- Class **DatFile**
This class is derived from class FileType. It reads necessary data from *dat* - file, which is produced by CAPAPRE.
- Class **Material**
This class contains methods for reading information about material from special file, which contains data about it. In future we plan to implement here also methods for the interpolation / approximation of nonlinear material behaviour, transformations of material tensors as well as the handling of inhomogeneous material data.
- Class **TimeFunc**
This class stores information about given time function. Later it should provide methods for handling time functions, interpolation (1D, 2D, 3D) of initial and load (nodal-, surface-, volume-type) data.
Input: FileType
- Class **OutResultUnverg**
Organization of classes for output result is the same as for input data. This class is derived from a base class OutResult. It provides methods for output result in *unverg* - format.
Input: Driver

4.2. Classes for handling grids on calculation domain

- Class **Grid**
This class contains methods for working with grid. For each level of refinement we

create structure *GridHierarchy*, which stores all information about grid, such as maximum number of elements and nodes, type of each element, its connection, coordinates of nodes.

- **Class Domain**

This class contains information about calculation domain. In this class we define subdomains, which correspond to different material regions as well as equations and, correspondingly, can have different discretization. For each subdomain a object of class Grid is allocated. It's possible situation, when on the same physical domain we have different discretization (e.g. coupled field problem, different discretization for mechanic and magnetic field), therefore different objects of class Grid for the same subdomain should be created.

4.3. Classes for defining finite element spaces

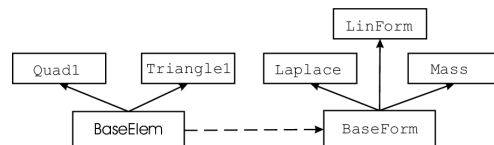


Fig. 2. Interaction between classes BaseElem and BaseForm

- **Class BaseElem**

This is a base class for description of finite element. In this class we define following

attributes of finite element: dimension, number of nodes, number of edges, number of faces, number of integration points, integration points themselves and appropriate integration weights. It stores virtual function for initialization information about finite element, set integration points, calculation of shape function and its derivatives at these points. These functions are substituted by specific functions in derived classes.

- **Classes Quad1, Triangle1, Quad2, Triangle2**

These classes are derived from BaseElem and contained specific information about , accordingly, bilinear quadrilateral, linear triangle, quadratic quadrilateral, quadratic triangle element and so on.

4.4. Classes for calculation the element matrices

- **Class BaseForm**

This class is a base class for computing the element matrices in accordance with form

$$\int \mathbf{B}^T \mathbf{D} \mathbf{B} \, dx$$

, where $\mathbf{B}$ is differential operator and $\mathbf{D}$ the material tensor. It provides methods for computation of element matrix on given element, type of which we define in constructor through the pointer to BaseElem. Main members of this class are functions for computing element matrix for different combination of shape function and its derivatives. For this purpose auxiliary functions, such as calculation of transformation given element in standard one, calculation of determinate of this transformation, are implemented. The virtual function CalcElemMatrix in this class is overloaded in each derived class.

Input: BaseElem

Output: Assemble

- **Class LaplaceInt**

This class is derived from class BaseForm for computation element stiffness matrix of laplace operator. This can be done by using main member of this class CalcElemMatrix.

- **Class MassInt**

This class is analogous to class LaplaceInt, only method CalcElemMatrix computes element mass matrix.

4.5. Classes for assembling and solving system matrix

- **Class SystemMatrix**

This class stores system matrix and right hand side arising from finite element discretization. It is base class for classes LinSystem and Assemble. It is done that way

in order to store these data once in program and there is explicit access to them from methods of classes `LinSystem` and `Assemble`. This class is implemented as template. Owing to that fact, we can define type of system matrix(sparse, symmetric or band matrix), when we determine object of the class.

- **Class `Assemble`**

This class provides methods for assembling global system matrix according to the weak form of the differential equations.

Input: Grid

- **Class `LinSystem`**

This class provides methods for solving linear algebraic system. At present Conjugate Gradiante method and BiCGSTAB method are implemented. This class is derived from class `SystemMatrix`, so it works with matrix and right hand side, which are stored in it.

- **Class `WorkWithSystemMatrix`**

This class is derived from class `LinSystem` and `Assemble`. It is created only with aim to have for classes `LinSystem` and `Assemble` the same base class `SystemMatrix`.

4.6. Specific classes for regarding problem

- **Class `Driver`**

The main element in the code organization is class `Driver`. It is a base class. In this class we conduct time-stepping. Due to this fact we conduct error estimation and refinement of grid in this class. In addition, the special algorithms concerned the coupling procedure of the different single field problems will be handled thereby. Constitutive action of this class are initialization derived classes from PDE in accordance with regarding problem, for example `AcousticPDE`, and define grid, on which this differential equation should be solved. Additionally, in this class we cause object of class `OutResultUnverg` for output of resultes.

Input: FileType, Material

Output: OutResultUnverg

- **Class `PDE`**

Class `PDE` is base class, from which classes for different type of PDE are derived. This class contains general things for different types of equations, for example, data about integration parameters, method to calculate parameters for Newmark method.

Input: FileType, Material

- **Class `AcousticPDE`**

This class is derived from class `PDE`. It is used for solving acoustic equation on one

time step. In this class we work with complex `WorkWithSystemMatrix`. When we create object of this class, we determine what kind of matrix should be system matrix, for example, sparse matrix, band matrix, symmetric matrix. We set rules for assembling global system matrix according to weak form of partial differential equation, define right hand side and set boundary conditions. Then we cause one of methods of `LinSystem` for solving linear system. On the last step we calculate first and second derivatives of the solution.

Input: Grid, TimeFunc

Output: `WorkWithSystemMatrix`

4.7. Useful tools

- Class **Vector**

This class is defined for working with arrays of different data types. It stores pointer to allocated array and its size. A lot of functions, such as multiplication, algebraic operation, resize, cut element, put element and so on, are implemented. Merit of this class is that we needn't deallocate arrays ourselves, this job would be done automatically by destructor. This class is implemented as simple template class with parameter - data type.

- Class **AbsMatrix**

It is a base class for different types of matrices. Principles of organization of storage matrix is similar for all types of matrices. The data of a matrix are allocated as a one contiguous memory block and there are pointers to the first element of each row. At present the following types of matrix are implemented:

- full matrix (class **Matrix**),
- symmetric matrix (class **SymMatrix**),
- sparse matrix (class **SparseMatrix**),
- symmetric sparse matrix (class **SymSparseMatrix**),
- band matrix (class **BandMatrix**),
- symmetric band matrix (class **SymBandMatrix**),

All these classes are derived from `AbsMatrix`. Each of these classes contains overloading algebraic operations, operation for output results in file or screen, cut or put elements in matrix, methods for resize and allocation matrix. Additionally, such function as precondition of matrix, is put in these classes. Due to this fact class `LinSystem` is independent from type of matrix, so we can use it as parameter in template of its. At present Jacobi, SSOR precondition type are implemented. For sparse matrix we use CRS(Compressed Row Storage) storage scheme. Lately we discuss one detail of implementation of hierarchy of these classes.

- Class **Clock**
This class provides tools for timing program.
- Class **Error**
Members of this class are used for informing user about mistake during runtime of program.

5. Some excerptes from code

5.1. Structure of the class Driver

```
void Driver::SolveNewmarkMethod(OutResultUnverg * ptUnverg)
{
    ptgrid=new Grid<Point2D>(ptFileType); // initialize grid for domain

    ptUnverg->Grid(ptgrid); // print information about grid in
                           // unverg-file

    ptAPDE=new AcousticPDE(...ptGrid, ptMaterial, ptFileType); //

    for (step=0; step<=numsteps; step++)
    {
        ptAPDE->SolveNewmarkMethodStatic(time);

        ptUnverg->OutputResult(ptAPDE->solution());
    }
}
```

5.2. Details of implementation of class AcousticPDE

```
\\ Constructor
AcousticPDE::AcousticPDE(Grid<Point2D> * ptgrid, Material * aptMaterial,
                        FileType * aptFileType, ...)
:PDE(aptFileType, aptMaterial)
// in PDE we initialize pointer to input file
// and pointer to file with material data
{
    // set time function
    ptTimeFunc=new TimeFunc(aptFileType);
    // read material info
    ptMaterial->ReadDensityAndCompress(density,compress); // read material info

    ...
    ptWork=new WorkWithSysMat<Point2D, Matrix<Double> >(ptGrid);

    ptWork->AssembleSysMatrix(CoefLaplace,CoefMass);
    ptWork->SetRHS();

    ptWork->SetNodesBoundaryCondition(aptFileType);
    ptWork->SetDirichletBoundaryCondSysMat_PenaltyMethod();
}
```

```
void AcousticPDE::SolveStatic_NewmarkMethod(const Double time, const Double dt)
{
    CalcParamForNewmarkMethod(dt);

    Boolean NeedRestore=FALSE;
    Double valueTF=ptTimeFunc->TimeFuncAtTime(time);

    if (valueTF==0)
    { NeedRestore=TRUE;
      ptWork->SetDirichletBoundaryCondZero_Cut();
    }
    else
      ptWork->SetDirichletBoundaryCondRHS_PenaltyMethod(valueTF);

    ptWork->CG(100, Jacobi);

    if (NeedRestore) ptWork->Restore();

    ... calculation of first and second derivatives of solution

    ... calculation of vectorM

    ptWork->SetRHS(VectM);

    solution=ptWork->getSolution();
}
```

5.3. Details of implementation of class **WorkWithSysMatrix**

```
template<class Dim, class T_Matrix>
template <class typeBaseForm>
void Assemble<Dim,T_Matrix>::AssembleGlobal(T_Matrix & Mat,BaseElem *)
{
    typeBaseForm oElemMatrix(ptElem,1);

    Dim * ptCoord=new Dim[numnodeelem]; // Dim: Point2D or Point3D

    for (numelem=0; numelem<maxnumelem; numelem++) // loop over all elements
    {
        ptgrid->GetConnection(connection,numelem);
        ptgrid->GetCoordOfNodesElem(numelem,ptCoord);

        for (ii=0; ii< MaxNumNodeElem; ii++)
            for (iii=0; iii< MaxNumNodeElem; iii++)
            {
                irow=connection[ii];
                icln=connection[iii];
                Mat(irow-1,icln-1)+=elemmat[ii][iii];
            }
    }
}
```

```

template<class Dim, class T_Matrix>
void Assemble<Dim,T_Matrix>::AssembleSysMatrix(const Double CoefL, const Double
CoefM)
{
    if (!IsCalcS) {
        AssembleGlobal< LaplaceInt<Dim> >(S); // S and M are stored
        IsCalcS=TRUE;                        // in data of this class
    }
    if (!IsCalcM) {
        IsCalcM=TRUE;
        AssembleGlobal< MassInt<Dim> >(M);
    }
    if (CoefM!=1.0) A+=M*CoefM;
    else A+=M;

    if (CoefL!=1.0) A+=S*CoefL;
    else A+=S;
}

```

```
void SetRHS(const Vector<Double> & CoefM=Vector<Double>(),
           const Vector<Double> & CoefS=Vector<Double>(),
           const Vector<Double> & f=Vector<Double>());

template<class Dim, class T_Matrix>
void Assemble<Dim, T_Matrix>::SetRHS(const Vector<Double> & CoefM,
                                     const Vector<Double> & CoefS, const Vector<Double> & f)
{
    if (!CoefM.size() && !CoefS.size() && !f.size()) b.Init();
    if (CoefM.size()) b=M*CoefM;
    if (CoefS.size())
        if (CoefM.size()) b+=S*CoefS;
        else b=S*CoefS;
    if (f.size()) if (!CoefM.size() || !CoefS.size()) b+=f;
                  else b=f;
}
```

```
template <class T_Matrix>
class SystemMatrix
{
protected:
    T_Matrix A; // system matrix

    ...
}

template< class Dim, class T_Matrix>
class WorkWithSysMat: virtual public Assemble<Dim,T_Matrix>,
                    virtual public LinSystem<Double, T_Matrix>;
```

5.4. Details of implementation of class Matrix

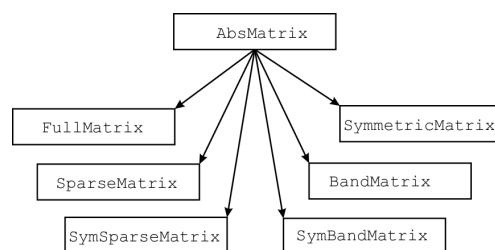


Fig. 3. Classes, which are derived from base class AbsMatrix

Use of mechanism of virtual function for the implementation of the inheritance-hierarchy for different types of matrices can ruin any algorithm, because the virtual functions are called too often. Good solution for this problem is approach "recursive template". The trick is that the base class takes a template parameter, which is the type of the leaf class. So at compile time complete type of an object is known. In this approach, leaf classes can have methods which are unique to the leaf class, and at the same time we can have default implementations of some methods.

```
template<class T_leaftype, class TYPE>
class AbsMatrix
{
public:
    T_leaftype & asLeaf()
    { return static_cast<T_leaftype*>(*this); }

    Integer getSize() { return asLeaf.GetSize();}

    Boolean IsSymmetric() { return FALSE;}
};

template<class TYPE>
class Matrix: public AbsMatrix<Matrix<TYPE> , TYPE>;

// Example routine which takes any matrix type
template< class T_leaftype, TYPE>
TYPE Spur(AbsMatrix<T_leaftype,TYPE> A);
```
